

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



Thành viên:

21120369 - Nguyễn Minh Vũ

21120558 - Phùng Hoài Thi

Báo cáo thực hành:

**BẢO MẬT TRONG QUÁ TRÌNH HUẤN
LUYỆN MÔ HÌNH MÁY HỌC**

**CSC18101 – Trí tuệ nhân tạo cho an ninh
thông tin**

Giảng viên thực hành:

ThS. Nguyễn Thị Hường

Thầy Nguyễn Ngọc Toàn

Tp. Hồ Chí Minh, tháng 12/2024

Mục lục

1	Nội dung thực hành	4
2	Điều chỉnh mô hình	5
3	Mô phỏng tấn công	7
3.1	Data Poisoning	7
3.2	Model Poisoning	10
3.3	Thử nghiệm tấn công	12
4	Phương Pháp Phòng Thủ	13
4.1	Phương Pháp: Mean Trimmed	13
4.2	Phương Pháp: L2 Regularization	14
4.3	Phương Pháp: Cosine Similarity	15
4.4	Thử nghiệm	17

Danh sách hình

2.1	Tổng hợp Loss của các Client qua quá trình huấn luyện với 10 epochs.	5
2.2	Kết quả huấn luyện tại Server	6
3.1	Tổng hợp Loss của các Client qua quá trình huấn luyện với 10 epochs.	9
3.2	Kết quả huấn luyện tại Server khi có tấn công dữ liệu . . .	9
3.3	Tổng hợp Loss của các Client qua quá trình huấn luyện với 10 epochs.	11
3.4	Kết quả của model poisoning lên máy chủ	11
4.1	Minh họa Mean Trimmed	13
4.2	Minh họa Cosine Similarity	15
4.3	So sánh Accuracy khi áp dụng kiểu tấn công Data Poisoning vào Client 2 khi có phòng thủ Trim	18
4.4	So sánh Accuracy khi áp dụng kiểu tấn công Model Poisoning vào Client 2 khi có phòng thủ Trim-L2	18

Danh sách bảng

1.1	Bảng đánh giá công việc	4
3.1	Accuracy và Loss khi áp dụng các tấn công vào mô hình .	12
4.1	Accuracy và độ Loss khi áp dụng các phương pháp phòng thủ.	16
4.2	Accuracy và Loss khi áp dụng phòng thủ trước các tấn công vào mô hình	17

Nội dung thực hành

Yêu cầu	Phụ trách	Mức độ hoàn thành
Điều chỉnh mô hình đạt hiệu suất $\geq 70\%$	Vũ	100%
Mô phỏng kiểu tấn công Data Poisoning	Vũ	100%
Mô phỏng kiểu tấn công Model Poisoning	Vũ	100%
Mô phỏng phòng thủ tại Server	Thi	100%

Bảng 1.1: Bảng đánh giá công việc

Điều chỉnh mô hình

Trong Lab02, nhóm đã sử dụng mô hình ResNet50 từ thư viện keras với mục đích trích xuất đặc trưng của ảnh từ đó nâng cao hiệu suất của mô hình hồi quy với hiệu năng đạt $\approx 70\%$. Bên cạnh đó, để tăng thêm hiệu suất, nhóm sử dụng thêm phương pháp UpSampling tăng kích thước của ảnh ban đầu với $param = 3$ tức tăng size ảnh lên gấp 3 lần ($32 \times 32 \times 3 \rightarrow 96 \times 96 \times 3$).

Kết quả huấn luyện với mô hình học liên kết:



(a) Loss của Client 0

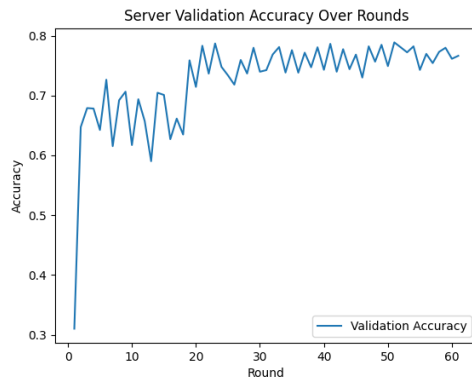


(b) Loss của Client 1

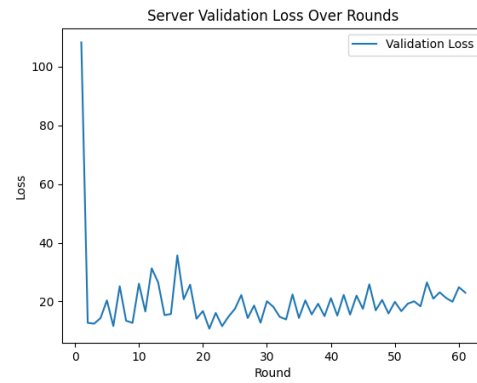


(c) Loss của Client 2

Hình 2.1: Tổng hợp Loss của các Client qua quá trình huấn luyện với 10 epochs.



(a) Accuracy của Server



(b) Loss của Server

Hình 2.2: Kết quả huấn luyện tại Server

Ở hình 2.1a, Client 0 được huấn luyện trên tập dữ liệu gồm 5 lớp (máy bay, ô tô, chim, mèo, hươu) trong tổng số 10 lớp nên khi tính toán loss trên toàn tập dữ liệu kết quả cho thấy loss tương đối cao, dao động từ 100 – 240. Tương tự với Client 1 ở hình 2.1b có độ loss khoảng 50 khi huấn luyện trên tập dữ liệu 9/10 lớp. Cuối cùng là Client 2 hình 2.1c được huấn luyện trên cả 10 lớp nên độ loss thấp khoảng 20.

Sang phía Server, ta có thể thấy mô hình học liên kết tại Server đạt $\text{Accuracy} \approx 80\%$ và có Loss trung bình ≈ 20 . Đây cũng sẽ là kết quả mà nhóm dựa vào để tiến hành các mô phỏng tấn công và phòng thủ.

Mô phỏng tấn công

3.1 Data Poisoning

Data Poisoning là cách tấn công mà người tấn công (malicious user) là 1 trong các client được kết nối với mô hình học liên kết. Khi này, người tấn công có khả năng làm sai lệch dữ liệu huấn luyện ở local dẫn đến mô hình huấn luyện đạt kết quả kém và truyền trọng số kém đó cho máy chủ tổng hợp (server) làm ảnh hưởng đến việc tính toán trọng số chung của máy chủ. [3]

Cụ thể hơn, nhóm đã mô phỏng mô hình tấn công với mục đích chính là gán nhãn sai lệch cho dữ liệu huấn luyện. Ví dụ, trong tập dữ liệu CIFAR-10, mô hình tấn công làm thay đổi nhãn của ảnh với nhãn "máy bay" thành "con ếch". Điều này diễn ra một cách ngẫu nhiên. Dưới đây là cách nhóm đã cài đặt cho kiểu mô phỏng tấn công này:

```
1 def poison_dataset(x_train, y_train, num_labels_to_poison=3, poison_ratio=0.1):
2     """
3     Poison the training dataset by randomly changing a fraction of samples for
4     multiple labels.
5
6     Args:
7         x_train (numpy.ndarray): Training images.
8         y_train (numpy.ndarray): Training labels.
9         num_labels_to_poison (int): Number of unique labels to poison.
10        poison_ratio (float): Fraction of the dataset to poison.
11
12    Returns:
13        Tuple[numpy.ndarray, numpy.ndarray]: Poisoned training images and labels.
14    """
15    unique_labels = np.unique(y_train)
16    labels_to_poison = np.random.choice(unique_labels, num_labels_to_poison,
17    replace=False)
18
19    y_train_poisoned = np.copy(y_train)
20    x_train_poisoned = np.copy(x_train)
```

```

21
22     for target_label in labels_to_poison:
23         # Choose a random desired label for the target label
24         desired_label = np.random.choice(unique_labels[unique_labels != target_label])
25
26         # Find indices of the target label
27         target_indices = np.where(y_train == target_label)[0]
28         num_available = len(target_indices)
29
30         # Determine how many samples to poison
31         num_poison = int(len(y_train) * poison_ratio / num_labels_to_poison)
32         num_poison = min(num_poison, num_available)
33
34         if num_poison > 0:
35             poison_indices = np.random.choice(target_indices, num_poison,
36                                               replace=False)
37             y_train_poisoned[poison_indices] = desired_label
38     print(f"Poisoned dataset with labels: {labels_to_poison} with poison ratio:
39           {poison_ratio}")
40     return x_train_poisoned, y_train_poisoned

```

Nhóm tiến hành mô phỏng tấn công vào tập dữ liệu của Client 2 với việc cố tình đánh nhãn sai cho các nhãn 3, 5, 9 hay "mèo, chó và xe tải" với tỉ lệ 50%, nghĩa là nếu tổng số lượng mẫu của các nhãn này trong Client là 3000 thì số nhãn bị đánh tráo là 1500. Dưới đây là kết quả nhóm thu được sau khi áp dụng mô phỏng tấn công vào mô hình học liên kết:

Chú ý ở hình 3.1c, ta thấy độ lỗi của Client 2 tăng lên đáng kể so với 2.1c vì dữ liệu đã bị tấn công bởi người dùng. Điều này dẫn đến Accuracy của Server 3.2a giảm tương đối đáng kể (chỉ còn $\approx 0.65\%$) so với ban đầu. Bên cạnh đó, các giá trị Accuracy và Loss của Server qua từng Round cũng rất hỗn loạn và gần cuối biểu đồ, ta cũng phát hiện rằng các giá trị có dấu hiệu bất thường. Đó là bởi vì 2 Client 0 và 1 ít dữ liệu hơn nên huấn luyện nhanh hơn và về cuối chỉ còn mô hình của Client 2 mà Client 2 lại bị tấn công nên các kết quả cho ra là bất thường.



(a) Loss của Client 0

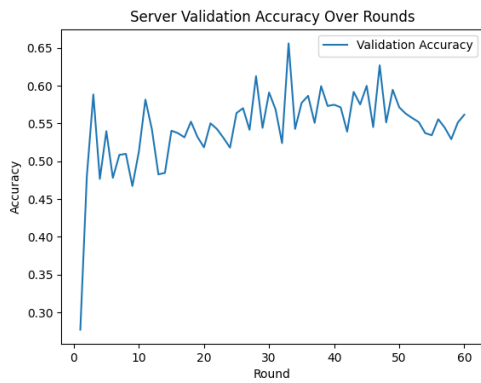


(b) Loss của Client 1

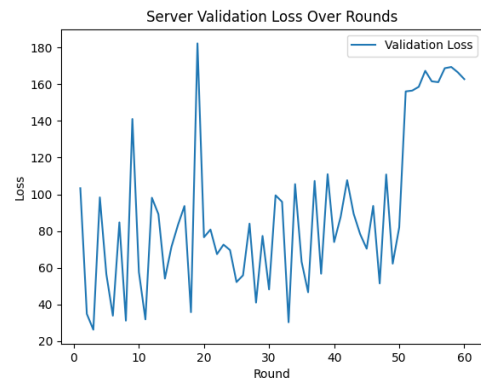


(c) Loss của Client 2

Hình 3.1: Tổng hợp Loss của các Client qua quá trình huấn luyện với 10 epochs.



(a) Accuracy của Server



(b) Loss của Server

Hình 3.2: Kết quả huấn luyện tại Server khi có tấn công dữ liệu

3.2 Model Poisoning

Tiếp theo, ta sẽ tìm hiểu về một kiểu tấn công khác mà tại đó, người tấn công thay vì xâm nhập vào trong dữ liệu thì sẽ cố tình làm sai lệch các tính toán trọng số tại Client trước khi gửi lên Server. Server khi nhận được trọng số này sẽ tổng hợp và làm ảnh hưởng chung đến cả quá trình huấn luyện học liên kết. Dưới đây là phần mã nguồn nhóm cài đặt cho phần mô phỏng tấn công vào mô hình. Hàm này đã được cài đặt ở Lab02 và chỉ thêm chức năng mới là đánh dấu đây là Client tấn công hay không [2]:

```
1 def _synchronize_weights(self, fake=False, attack_strength=0.5):
2     """
3     Sends local model weights to server and receives aggregated weights.
4     Optionally applies a model poisoning attack.
5
6     Args:
7         fake (bool): Whether to apply model poisoning (malicious behavior).
8         attack_strength (float): Strength of the model poisoning attack.
9     """
10    print('\nSynchronizing weights...')
11    with self._connect_to_server() as worker_socket:
12        # Send client index
13        self._send_np_array([self._client_index], worker_socket)
14
15        # Optionally poison the weights
16        if fake:
17            poisoned_weights = []
18            for weight in self.model.get_weights():
19                if np.issubdtype(weight.dtype, np.floating):
20                    noise = np.random.normal(0, attack_strength, weight.shape)
21                    poisoned_weights.append(weight + noise)
22                else:
23                    poisoned_weights.append(weight)
24            self._send_np_array(poisoned_weights, worker_socket)
25        else:
26            # Send unmodified weights
27            self._send_np_array(self.model.get_weights(), worker_socket)
28
29        # Receive aggregated weights from the server
30        broadcasted_weights = self._get_np_array(worker_socket)
31        self.model.set_weights(broadcasted_weights)
32        worker_socket.close()
```



(a) Loss của Client 0

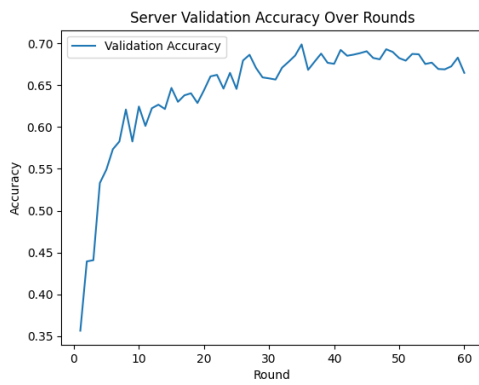


(b) Loss của Client 1

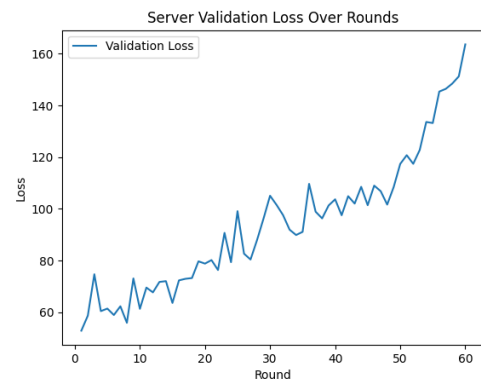


(c) Loss của Client 2

Hình 3.3: Tổng hợp Loss của các Client qua quá trình huấn luyện với 10 epochs.



(a) Accuracy của Server



(b) Loss của Server

Hình 3.4: Kết quả của model poisoning lên máy chủ

3.3 Thử nghiệm tấn công

Method	Client 0	Client 1	Client 2	Server
data0-012	<i>20.54%</i>	70.32%	69.36%	73.66%
	270.1454	35.4463	14.4910	74.3384
data1-012	<i>43.00%</i>	<i>54.70%</i>	68.30%	68.60%
	371.4643	107.2137	16.1385	110.6368
data2-012	<i>46.54%</i>	69.88%	<i>51.82%</i>	61.38%
	193.0082	42.1704	59.4833	121.9774
model0	<i>43.82%</i>	66.46%	71.80%	73.26%
	263.7077	40.5621	19.8870	95.4963
model1	<i>42.86%</i>	<i>65.62%</i>	68.94%	69.02%
	277.8181	54.4610	30.3182	130.6952
model2	<i>43.80%</i>	66.24%	<i>69.56%</i>	64.94%
	282.6458	59.7970	34.1861	150.1995

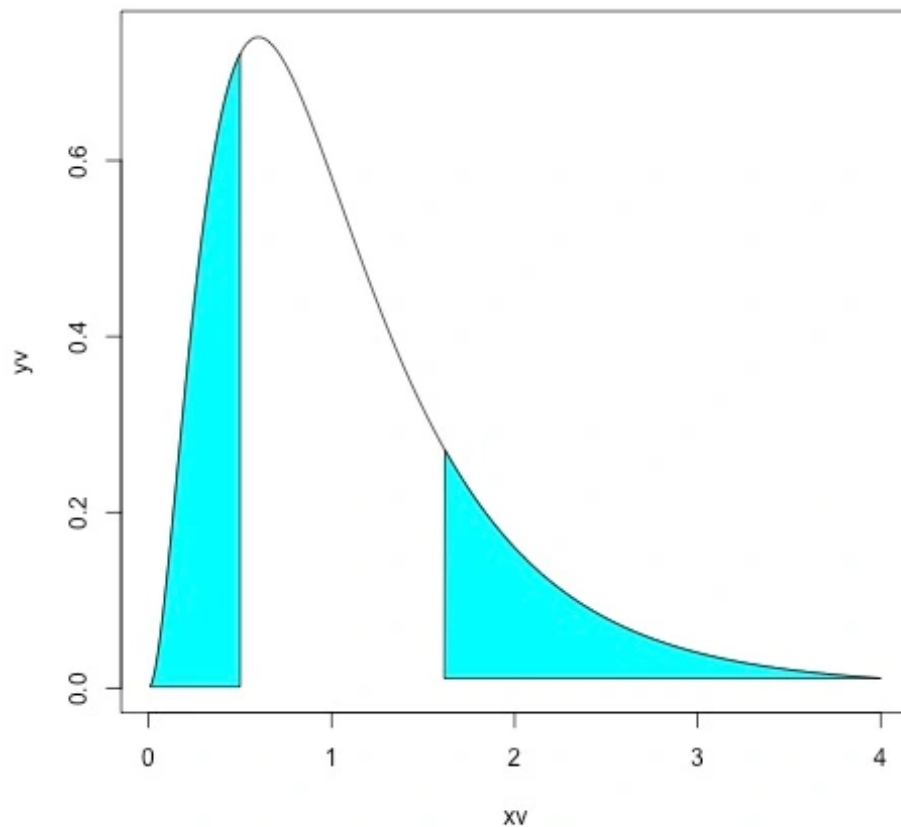
Bảng 3.1: Accuracy và Loss khi áp dụng các tấn công vào mô hình

Các phương pháp dùng thử nghiệm được kí hiệu như sau: data/model tượng trưng cho 2 kiểu tấn công data poisoning và model poisoning. Các số liền kề thể hiện áp dụng tấn công trên Client nào (data0 - data poisoning trên Client 0). Đối với kiểu tấn công data poisoning có thêm kí hiệu 012 chỉ các nhãn nào sẽ bị đánh tráo khi tấn công.

Ta thấy rằng các kết quả được thể hiện rõ rệt khi áp dụng tấn công, các mô hình bị tấn công có xu hướng giảm độ chính xác **màu đỏ** so với chính nó khi không bị tấn công. Bên cạnh đó, các kết quả tại Server cũng chỉ ra mức độ ảnh hưởng của các mô hình có tính thiên vị. Chẳng hạn với Client 0, kết quả cho ra cực kì thấp $\approx 20.54\%$ nhưng vì Client này có ít dữ liệu huấn luyện hơn nên không đóng góp nhiều vào trọng số chung của Server nên Accuracy của Server vẫn được giữ ở mức ổn định ($\geq 70\%$). Từ đó, ta có nhận xét về Accuracy của Server tương quan với mức độ thiên vị của Client đối với Server khi tấn công.

Phương Pháp Phòng Thủ

4.1 Phương Pháp: Mean Trimmed



Hình 4.1: Minh họa Mean Trimmed

Phương pháp *Mean Trimmed* là một kỹ thuật giúp giảm ảnh hưởng của các giá trị cực trị (outliers) trong dữ liệu mà không phải loại bỏ hoàn toàn các điểm dữ liệu từ client. Cụ thể, phương pháp này sẽ lọc và loại bỏ một phần nhỏ các giá trị dữ liệu có sự sai lệch quá mức (ví dụ, các giá trị cực lớn hoặc cực nhỏ) trong quá trình tính toán trung bình.

Cách thức hoạt động:

- Đầu tiên, các client gửi các cập nhật của họ tới server. Server sẽ nhận tất cả các giá trị được cập nhật và tính toán giá trị trung bình từ các cập nhật này.
- Sau đó, server sẽ loại bỏ một tỷ lệ nhỏ (ví dụ: 5% hoặc 10%) các giá trị cực trị từ tập cập nhật. Những giá trị này có thể là các giá trị quá lớn hoặc quá nhỏ mà không phản ánh đúng phân phối chung của các client.
- Cuối cùng, server tính toán lại giá trị trung bình từ các cập nhật còn lại và sử dụng giá trị này để cập nhật mô hình.

Phương pháp này giúp bảo vệ mô hình khỏi các điểm dữ liệu bị tấn công mà không loại bỏ hoàn toàn các client. Việc chỉ loại bỏ các giá trị sai lệch giúp duy trì độ chính xác của mô hình mà không làm mất quá nhiều dữ liệu hữu ích từ các client không bị tấn công. Trong trường hợp có ba client, phương pháp này không làm giảm thiểu thông tin quan trọng từ các client hợp lệ, mà chỉ giảm thiểu ảnh hưởng từ các điểm dữ liệu sai lệch.

4.2 Phương Pháp: L2 Regularization

L2 Regularization (hay còn gọi là *Ridge Regularization*) là một phương pháp điều chuẩn phổ biến giúp làm giảm sự ảnh hưởng của các trọng số lớn trong mô hình. Khi một client bị tấn công và cố gắng thay đổi trọng số một cách sai lệch, phương pháp này sẽ giúp giữ cho các trọng số của mô hình không bị thay đổi quá mức, từ đó giảm thiểu tác động của các cập nhật nhiễu.

Cách thức hoạt động:

- Hàm mất mát trong học máy có thể được biểu diễn như sau:

$$L = L_{\text{original}} + \lambda \sum_{i=1}^n w_i^2$$

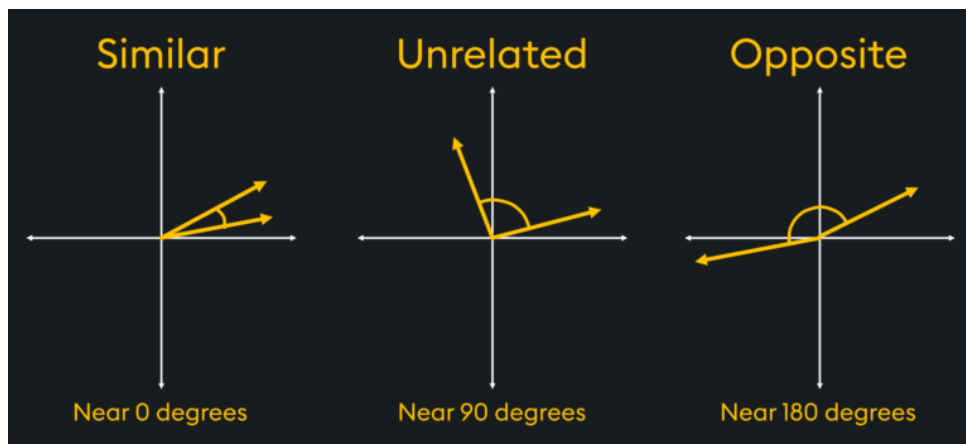
trong đó:

- L_{original} là hàm mất mát gốc (như hàm lỗi trong mô hình học).

- w_i là trọng số của mô hình.
- λ là hệ số điều chuẩn, điều chỉnh độ mạnh của việc áp dụng điều chuẩn L2.
- Việc thêm phần điều chuẩn này giúp giảm giá trị trọng số, ngăn ngừa các trọng số quá lớn từ các client có thể gây hại cho mô hình.

Khi có sự tấn công vào mô hình, các client có thể cập nhật trọng số không chính xác hoặc trái ngược với các client khác. Phương pháp *L2 Regularization* sẽ tác động trực tiếp vào trọng số của mô hình, hạn chế sự thay đổi quá mức và giữ cho các trọng số ổn định hơn. Điều này giúp giảm thiểu sự ảnh hưởng của các cập nhật nhiễu từ các client bị tấn công.

4.3 Phương Pháp: Cosine Similarity



Hình 4.2: Minh họa Cosine Similarity

Cosine Similarity đo lường mức độ tương đồng giữa hai vector thông qua góc giữa chúng, thể hiện mức độ giống nhau giữa hai đối tượng. Trong trường hợp này, chúng ta tính toán sự tương đồng giữa các trọng số của các client và sử dụng thông tin này để điều chỉnh trọng số của các client khi tổng hợp các cập nhật.

Cách thức hoạt động:

- Tính cosine similarity giữa các client i và j thông qua công thức:

$$\text{sim}(i, j) = \frac{w_i \cdot w_j}{\|w_i\| \|w_j\|}$$

trong đó:

- w_i và w_j là các vector trọng số của client i và client j .
 - $\|w_i\|$ và $\|w_j\|$ là độ dài (norm) của các vector trọng số tương ứng.
- Sau khi tính toán cosine similarity giữa tất cả các client, ta sẽ ưu tiên các cập nhật từ những client có sự tương đồng cao với các client khác, từ đó giảm thiểu ảnh hưởng của các client bất thường hoặc bị tấn công.

Phương pháp *Cosine Similarity* giúp nhận diện và ưu tiên các cập nhật từ những client có mô hình gần giống với các client khác. Điều này đặc biệt quan trọng trong trường hợp có sự tấn công, vì nó giúp giảm thiểu ảnh hưởng từ các client có mô hình bị tấn công và ưu tiên những client có sự tương đồng cao với mô hình chung của hệ thống. [1]

Defense	Client 0	Client 1	Client 2	Server
mean	47.86%	72.16%	69.90%	74.22%
	184.1569	24.5191	15.9685	61.8961
trim	47.48%	71.64%	78.44%	78.42%
	138.7201	41.7406	19.8166	26.2839
triml2	39.76%	61.40%	74.26%	75.38%
	110.6644	21.3976	15.0574	9.9881

Bảng 4.1: Accuracy và độ Loss khi áp dụng các phương pháp phòng thủ.

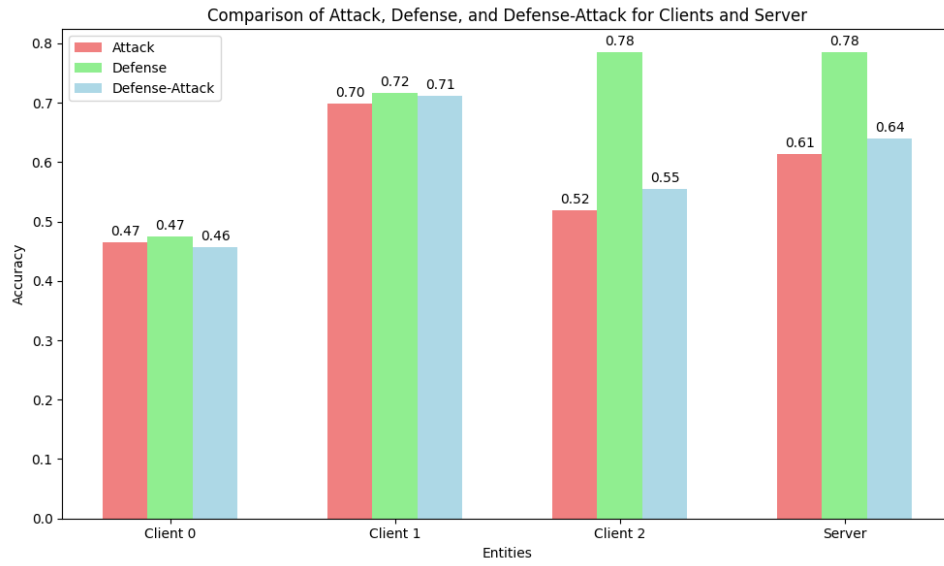
Kết quả từ Server cho thấy khi áp dụng các phương pháp phòng thủ khác nhau, ta có thể đảm bảo độ chính xác (đôi khi tăng hơn so với không áp dụng).

4.4 Thử nghiệm

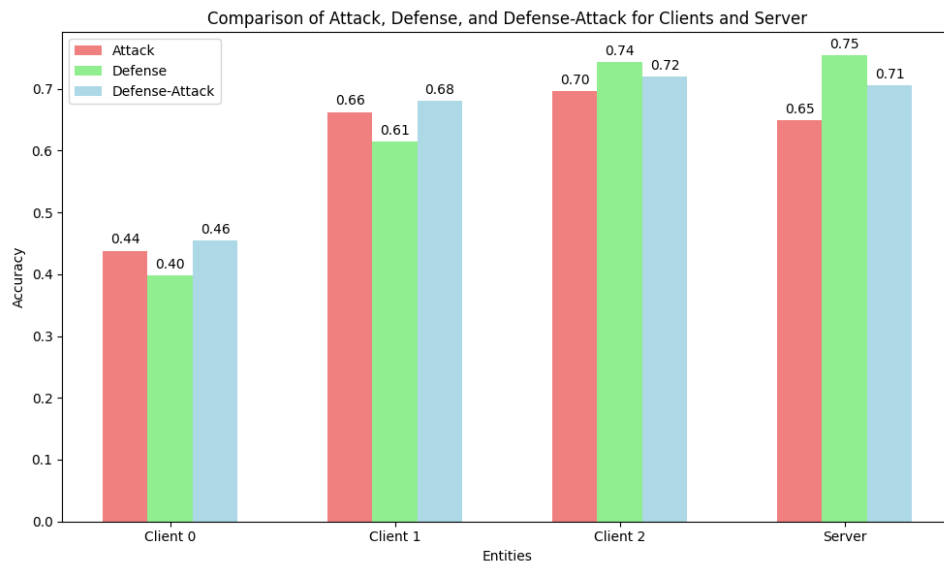
Method	Client 0	Client 1	Client 2	Server
Hyperparameter: L2 = 0.00005, Trim = 0.05				
data0-012-trim	<i>21.26%</i>	<i>68.98%</i>	<i>78.44%</i>	<i>78.28%</i>
	211.3589	35.1035	19.9106	35.3388
data1-012-triml2	<i>20.54%</i>	<i>73.22%</i>	<i>78.68%</i>	<i>78.60%</i>
	307.2370	36.6946	18.6949	35.5304
data2-012-trim	<i>43.50%</i>	<i>57.42%</i>	<i>78.32%</i>	<i>78.26%</i>
	361.0038	145.2158	18.8239	50.6362
model0-triml2	<i>42.64%</i>	<i>69.36%</i>	<i>75.68%</i>	<i>74.68%</i>
	271.8670	59.0393	40.5174	60.7272
model1-triml2	<i>43.20%</i>	<i>68.84%</i>	<i>73.74%</i>	<i>73.28%</i>
	276.4293	100.0227	65.4308	91.2567
model2-triml2	<i>45.50%</i>	<i>68.06%</i>	<i>72.02%</i>	<i>70.60%</i>
	258.1499	90.4595	74.3402	99.1544

Bảng 4.2: Accuracy và Loss khi áp dụng phòng thủ trước các tấn công vào mô hình

So với 3.1, ta có thể thấy các kết quả đã được tăng lên $\geq 70\%$ khi đưa phòng thủ vào cho Server. Điều này đảm bảo cho Server luôn đạt hiệu suất ổn định mặc cho các tấn công vào các Client.



Hình 4.3: So sánh Accuracy khi áp dụng kiểu tấn công Data Poisoning vào Client 2 khi có phòng thủ Trim



Hình 4.4: So sánh Accuracy khi áp dụng kiểu tấn công Model Poisoning vào Client 2 khi có phòng thủ Trim-L2

Tài liệu tham khảo

- [1] Nick Galanis. *Defending against Data Poisoning Attacks in Federated Learning via User Elimination*. 2024. arXiv: 2404.12778 [cs.CR]. URL: <https://arxiv.org/abs/2404.12778>.
- [2] Kai Li et al. *Data-Agnostic Model Poisoning against Federated Learning: A Graph Autoencoder Approach*. 2023. arXiv: 2311.18498 [cs.LG]. URL: <https://arxiv.org/abs/2311.18498>.
- [3] Gan Sun et al. “Data poisoning attacks on federated machine learning”. In: *IEEE Internet of Things Journal* 9.13 (2021), pp. 11365–11375.